Documentación del sistema Tienda en Python

Manual de usuario y documentación técnica

Alumno: Diego Alessandro Corona Beltran

Introducción

Tienda es una aplicación de escritorio para una pequeña papelería. Permite registrar clientes y productos, controlar existencias, realizar ventas y generar una nota digital con subtotal, IVA del 16 % y total. El inicio de sesión distingue las funciones del administrador y del cajero.

Este documento explica cómo ejecutar y utilizar el sistema, cómo se organiza su código y cómo se conservan los datos. Incluye el desarrollo de la práctica, los resultados verificados, la discusión y el cuestionario. Está dirigido tanto a quien atiende la tienda como a quien revisa o mantiene el proyecto.

Objetivo general

Desarrollar y documentar una interfaz visual en Python para administrar la información de una tienda y registrar sus ventas, sin utilizar bases de datos, con permisos según el rol del usuario.

Objetivos específicos

Registrar y consultar clientes y artículos; validar los datos de entrada; calcular los importes con Decimal; descontar existencias al vender; conservar las operaciones en JSON; y emitir tickets digitales recuperables desde el historial.

El programa utiliza la biblioteca estándar de Python y Tkinter. El archivo INICIAR.bat es un acceso de ejecución para Windows y no contiene la lógica del sistema.

Alcance y requisitos del sistema

La solución se ejecuta localmente con una sola instancia sobre un archivo de datos. Las operaciones confirmadas se conservan al cerrar el programa. El carrito pertenece a la sesión actual y no se almacena como venta hasta confirmar el cobro.

Requisitos funcionales

Identificador	Función	Implementación
RF01	Identificar al usuario y su rol	Inicio y cierre de sesión.
RF02	Conservar datos de clientes	Alta, consulta, edición y baja lógica.
RF03	Conservar datos de productos	Catálogo con precios y existencias.
RF04	Registrar ventas	Carrito, cobro y descuento de inventario.
RF05	Calcular importes	Detalle, subtotal, IVA, total y cambio.
RF06	Generar nota digital	Ventana de ticket y exportación TXT.
RF07	Recuperar ventas anteriores	Historial con consulta del ticket.

Permisos de los roles

Operación	Administrador	Cajero
Consultar clientes y productos	Sí	Sí
Registrar clientes nuevos	Sí	Sí
Editar y dar de baja clientes	Sí	No
Registrar y editar productos	Sí	No
Dar de baja productos	Sí	No
Registrar ventas y consultar tickets	Sí	Sí

Decisiones de diseño

Los precios del catálogo no incluyen IVA. El sistema admite pagos en efectivo, cantidades enteras y una tasa fija del 16 %, conforme al enunciado. Los códigos identifican los registros y no se reutilizan después de una baja. Las ventas guardan copias del nombre y del precio para conservar su detalle histórico.

La entrega no incorpora facturación fiscal, devoluciones, cancelación de ventas, descuentos, reportes contables, administración de usuarios ni sincronización entre equipos. El ticket es una nota de venta académica.

Instalación e inicio de sesión

Preparación del equipo

Se requiere Python 3.10 o posterior con Tkinter y una carpeta donde el usuario pueda escribir. La aplicación se verificó en Windows con Python 3.12.14 y Tcl/Tk 8.6.12. La ventana inicial mide 1120 por 760 píxeles y admite redimensionamiento desde 980 por 700.

1. Extraer el ZIP completo y localizar la carpeta Tienda. Mantener juntos tienda.py y modelo.py.
2. Abrir INICIAR.bat en Windows. Como alternativa, abrir una terminal en esa carpeta y ejecutar python tienda.py.
3. Escribir el usuario y la contraseña y pulsar Iniciar sesión o la tecla Enter.

Usuario	Contraseña de demostración	Rol
admin	Admin123!	Administrador
cajero	Caja123!	Cajero

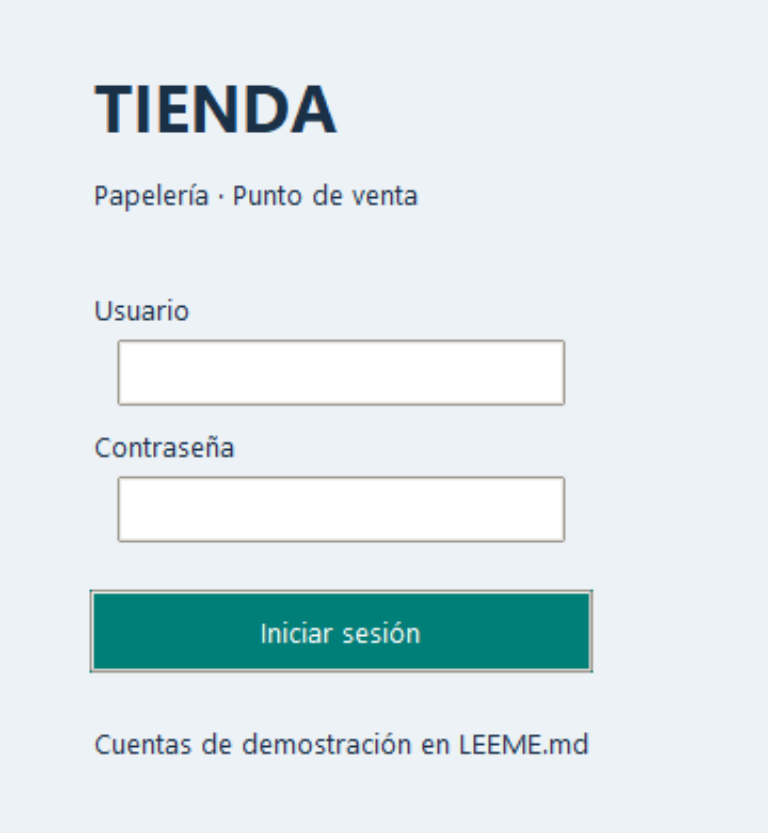


Figura 1 Detalle de la pantalla real de inicio de sesión

Las contraseñas distinguen mayúsculas y minúsculas. Un acceso incorrecto muestra Usuario o contraseña incorrectos y no abre la sesión. Las cuentas de esta versión son fijas.

Al primer inicio se crea datos/tienda.json con un cliente, cuatro productos y las cuentas de demostración. Para elegir otra ubicación se puede ejecutar python tienda.py --datos otra_carpeta/tienda.json.

Manual de clientes y productos

Registrar un cliente

1. Abrir la pestaña Clientes y pulsar Nuevo / Limpiar.

2. Escribir un código único y el nombre. Teléfono, correo y dirección son opcionales.

3. Pulsar Guardar. El registro aparece en la tabla y queda disponible en la lista de clientes de Nueva venta.

Ejemplo de alta: código C002 y nombre Ana Pérez. Si se escribe un correo, debe tener un formato básico como ana@example.com. El cajero puede registrar clientes nuevos; la edición y la baja requieren la cuenta de administrador.

Registrar un producto

1. Iniciar sesión como administrador y abrir Productos.

2. Pulsar Nuevo / Limpiar. Capturar el código, el nombre, el precio antes de IVA y las existencias.

3. Pulsar Guardar y comprobar el registro en la tabla.

Ejemplo de alta: código P005, nombre Regla de 30 cm, precio 15.00 y existencias 20. El precio debe ser positivo; las existencias admiten cero. Se escribe un punto para separar los decimales y no se incluyen símbolos de moneda ni separadores de miles.

Editar y dar de baja

Para editar, el administrador selecciona la fila del catálogo, cambia los datos y pulsa Guardar. El código se muestra como solo lectura. Nuevo / Limpiar abandona la selección y prepara el formulario para un alta.

Para retirar un registro, se selecciona la fila y se pulsa Dar de baja; después se confirma la acción. Se cambia activo a false y el registro deja de aparecer en las opciones disponibles. Las ventas anteriores permanecen. No existe una opción de reactivación en esta versión. El cliente C001 Público general no puede darse de baja.

Reglas de captura

Dato	Regla aplicada
Código	De 1 a 20 caracteres; letras A a Z, números, guion y guion bajo. Se convierte a mayúsculas.
Nombre	Obligatorio, de 1 a 80 caracteres y sin saltos de línea.
Datos opcionales del cliente	Hasta 120 caracteres por campo, sin controles ni saltos de línea.
Precio	Positivo después del redondeo; máximo de entrada 999999999.
Existencias	Número entero entre 0 y 999999.

Al refrescar el catálogo, los productos dados de baja o sin existencias se retiran del carrito; si se reduce el inventario de una línea pendiente, su cantidad se ajusta a la disponibilidad actual.

Manual de registro de ventas

1. En Nueva venta seleccionar el cliente. Público general está disponible para las compras que no requieren otro registro.
2. Elegir un código de producto, escribir la cantidad y pulsar Agregar. Repetir para cada artículo.
3. Revisar el detalle. Quitar seleccionado elimina una línea completa; Vaciar carrito retira todos los artículos.
4. Capturar el efectivo recibido y pulsar Cobrar y generar ticket. Revisar el total y confirmar la operación.

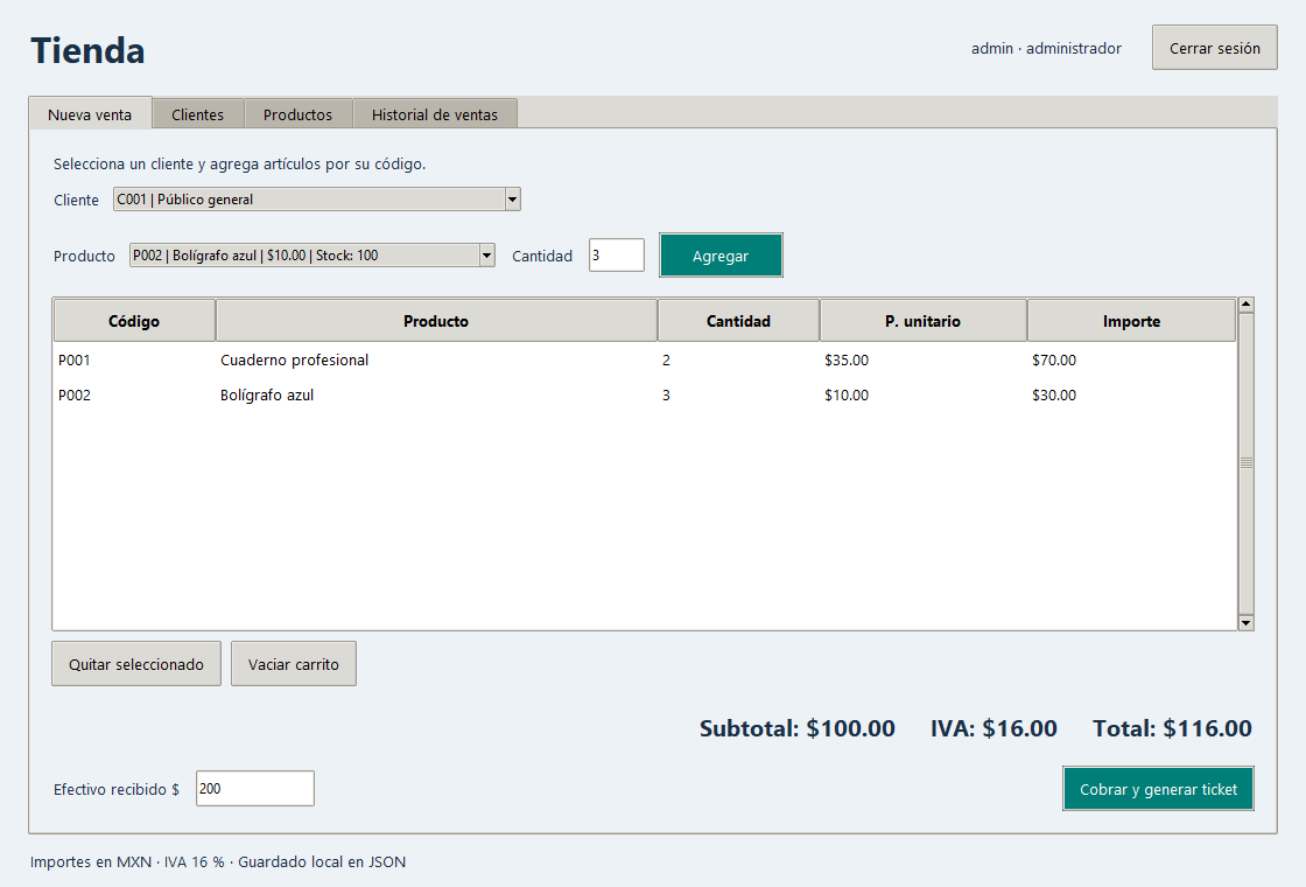


Figura 2 Venta de ejemplo antes de confirmar el cobro

Concepto	Operación	Resultado
Cuadernos P001	2 × $35.00	$70.00
Bolígrafos P002	3 × $10.00	$30.00
Subtotal	$70.00 + $30.00	$100.00
IVA	$100.00 × 0.16	$16.00
Total	$100.00 + $16.00	$116.00
Cambio con pago de $200.00	$200.00 - $116.00	$84.00

La venta se registra y las existencias se descuentan una sola vez al confirmar el cobro. En este ejemplo, P001 pasa de 50 a 48 unidades y P002 de 100 a 97. Cancelar la confirmación conserva el carrito pendiente.

Tickets y conservación de los datos

Consultar y guardar una nota de venta

Después del cobro aparece una ventana con folio, fecha, cliente, usuario que atendió, detalle de artículos, cantidades, precios unitarios, importes, subtotal, IVA, total, pago y cambio. Guardar ticket .txt permite elegir una carpeta y exportar el texto en UTF-8.

Para recuperar una nota, abrir Historial de ventas, seleccionar el folio y pulsar Ver / Guardar ticket. El historial presenta las operaciones de la más reciente a la más antigua. La venta se conserva en JSON aunque se cierre el ticket sin exportarlo; la aplicación no crea un TXT automáticamente.

```
Folio: V000001
Cliente: Público general (C001)
P001 | Cuaderno profesional
  2 x $35.00 = $70.00
P002 | Bolígrafo azul
  3 x $10.00 = $30.00
Subtotal: $100.00
IVA (16 %): $16.00
TOTAL: $116.00
Pago: $200.00
Cambio: $84.00
```

Cerrar sesión y respaldar

Cerrar sesión vuelve al acceso. Si hay un carrito pendiente, se solicita confirmar que se descarte; lo mismo ocurre al cerrar la ventana principal. Las ventas confirmadas ya están guardadas.

Para respaldar, cerrar todas las instancias y copiar la carpeta datos a otra ubicación. Para restaurar, cerrar la aplicación, conservar una copia del archivo actual y colocar el respaldo válido como datos/tienda.json. El siguiente inicio cargará ese archivo. Los tickets exportados se respaldan por separado en la carpeta elegida al guardarlos.

Resolver errores frecuentes

Mensaje o situación	Acción del usuario
Python no se reconoce	Revisar la instalación de Python y ejecutar desde una terminal. INICIAR.bat intenta py -3 y, si no existe, python.
Usuario o contraseña incorrectos	Revisar las credenciales y las mayúsculas. No hay recuperación de contraseña en la interfaz.
Existencias insuficientes	Reducir la cantidad o solicitar al administrador que revise el inventario.
Pago insuficiente o importe inválido	Introducir un número válido con punto decimal y un pago que cubra el total.
No se puede leer o guardar el archivo	Conservar el archivo. Revisar su contenido, los permisos y el espacio disponible; restaurar un respaldo si corresponde.

Documentación técnica y arquitectura

La aplicación separa dos responsabilidades: Aplicacion, en tienda.py, presenta las pantallas y atiende eventos; Tienda, en modelo.py, comprueba reglas y administra la persistencia. Los datos circulan como diccionarios y listas. No hay conexión de red ni motor de base de datos.

```
Controles de Tkinter
    -> Aplicacion en tienda.py
    -> Tienda en modelo.py
    -> datos/tienda.json

Venta guardada -> ticket(venta) -> ventana o archivo TXT
```

Archivos del proyecto

Archivo	Responsabilidad
tienda.py	Inicio, ventana principal, catálogos, carrito, cobro, historial y exportación.
modelo.py	Sesión, permisos, validación, operaciones del negocio y JSON.
test_tienda.py	13 pruebas automatizadas con unittest y datos temporales.
INICIAR.bat	Lanzador opcional de Windows; no implementa reglas del negocio.
LEEME.md y VERIFICACION.txt	Guía de ejecución y evidencia de verificación.
datos/tienda.json	Archivo generado al ejecutar; conserva registros y ventas.

Métodos principales del modelo

Método	Entrada y resultado
entrar y salir	Comprueban credenciales y establecen o limpian usuario y rol en memoria.
catalogo	Recibe clientes o productos y devuelve una copia de los registros activos.
guardar_cliente y guardar_producto	Reciben los campos del registro y editar; validan y guardan los cambios.
eliminar	Recibe tipo y código; aplica una baja lógica con autorización de administrador.
cotizar	Recibe el carrito y devuelve detalle, subtotal, IVA y total sin guardar la venta.
vender e historial	Confirma una venta o devuelve una copia de las ventas guardadas.
ticket	Función externa a la clase; transforma una venta en una cadena de texto.

proteger() captura ValueError, PermissionError y OSError para mostrar mensajes en la interfaz. refrescar() actualiza tablas y listas después de un cambio. cobrar() solicita confirmación antes de invocar vender().

Modelo de datos y procesamiento

El documento JSON contiene version, usuarios, clientes, productos y ventas. Clientes y productos se indexan por su código; el carrito usa la relación código a cantidad. Los importes persistidos son cadenas con dos decimales para conservar su representación exacta al convertirlos nuevamente a Decimal.

Entidad	Campos y tipos almacenados
Usuario	Clave: nombre de usuario. sal y hash: cadenas hexadecimales. rol: cadena.
Cliente	Clave: código. nombre, telefono, correo y direccion: cadenas. activo: booleano.
Producto	Clave: código. nombre y precio: cadenas. existencias: entero. activo: booleano.
Venta	folio, fecha, cliente_codigo, cliente y usuario: cadenas. detalles: lista. subtotal, iva, total, pago y cambio: cadenas.
Detalle	codigo y producto: cadenas. cantidad: entero. precio e importe: cadenas.

Secuencia para confirmar una venta

1. Verificar la sesión y comprobar que el cliente exista y esté activo.
2. Validar el carrito: debe tener productos activos, cantidades enteras positivas y existencias suficientes.
3. Calcular cada importe, sumar el subtotal, aplicar IVA del 16 % y validar el pago.
4. Asignar folio V seguido de seis dígitos, fecha con zona horaria local y usuario de la sesión.
5. Copiar el estado, descontar inventario y agregar la venta con su detalle histórico.
6. Escribir el JSON actualizado y, solo si la escritura tiene éxito, reemplazar el estado en memoria.

Dinero y escritura del archivo

dinero() valida valores finitos entre 0 y 999999999 y aplica dos decimales con ROUND_HALF_UP. El producto exige además un precio positivo después del redondeo. entero() acepta existencias de 0 a 999999 o cantidades vendidas de 1 a 999999. texto() rechaza campos vacíos, longitudes excesivas y caracteres de control.

_guardar() escribe en un archivo temporal de la misma carpeta, vacía el búfer, solicita fsync y reemplaza el archivo con os.replace. _confirmar() cambia self.datos después de guardar. Un error anterior a ese reemplazo evita confirmar la operación. Esto no sustituye un respaldo ni proporciona bloqueo entre procesos.

Protección y límites

Las contraseñas se derivan con PBKDF2 HMAC SHA-256, 120000 iteraciones y una sal aleatoria de 16 bytes por usuario. La comparación usa hmac.compare_digest. El archivo no está cifrado y quien pueda editarlo puede alterar datos o roles. Las cuentas de demostración también están definidas en el código; esta versión está destinada a uso académico local.

Resultados de pruebas y discusión

La verificación registrada para la entrega se ejecutó con Python 3.12.14 y Tcl/Tk 8.6.12. Las 13 pruebas del modelo finalizaron correctamente. Se ejecutan desde la carpeta del proyecto con el siguiente comando; crean archivos temporales independientes y no usan el archivo de trabajo del usuario.

```
python -m unittest -v test_tienda.py
```

Caso	Comportamiento verificado	Resultado
1	Alta, edición y baja de registros.	Correcto
2	Archivo JSON corrupto conservado.	Correcto
3	Credenciales y cierre de sesión.	Correcto
4	Duplicados y datos de captura inválidos.	Correcto
5	Fallo de escritura sin confirmar cambios.	Correcto
6	Importes no finitos y enteros inválidos rechazados.	Correcto
7	Copias de datos y ausencia de contraseña en claro en JSON.	Correcto
8	Persistencia y continuidad de folios al reabrir.	Correcto
9	Venta inválida rechazada sin cambiar el estado.	Correcto
10	Redondeo del IVA a centavos.	Correcto
11	Permisos y operaciones del cajero.	Correcto
12	Conservación del nombre y precio históricos.	Correcto
13	Cálculo del ticket y descuento de existencias.	Correcto

La comprobación adicional de interfaz abrió el acceso, inició sesión, revisó las cuatro pestañas, agregó productos, confirmó el cobro, mostró el ticket y revisó el historial y los controles del cajero. Las figuras corresponden a capturas de esa aplicación.

Resultados del caso de venta

Dos cuadernos de $35.00 y tres bolígrafos de $10.00 produjeron $100.00 de subtotal, $16.00 de IVA y $116.00 de total. Un pago de $200.00 produjo $84.00 de cambio. El ticket_ejemplo.txt conserva una muestra del resultado.

Discusión

La separación entre interfaz y modelo facilitó probar permisos, validación y dinero. JSON cumple el requisito de trabajar sin bases de datos en una instancia local; aumenta el trabajo de lectura y escritura al crecer el historial. La validación al cargar comprueba la versión y los tipos de las secciones principales, pero no revisa exhaustivamente cada campo interno.

El proyecto cubre los objetivos de la práctica. Para un uso operativo posterior harían falta, entre otras decisiones, control de acceso al archivo, administración de usuarios y protección ante escritura concurrente. Estas funciones no forman parte de la implementación documentada.

Cuestionario de la práctica

1. ¿Qué paradigmas de programación utiliza Python?

Es multiparadigma: admite programación imperativa y procedimental, orientación a objetos y programación funcional. En este proyecto se usan clases para organizar el modelo y la aplicación, instrucciones secuenciales para las operaciones y funciones como manejadores de eventos. [2]

2. ¿Cómo se clasifican los tipos de datos estándar?

Numéricos: int, float y complex; bool es un subtipo de int. Secuencias: list, tuple, range y str. Binarios: bytes, bytearray y memoryview. Mapeos: dict. Conjuntos: set y frozenset. NoneType representa la ausencia de valor con None. Python también dispone de tipos para funciones, clases y excepciones. [3]

3. Palabras reservadas y breve descripción

Listado de las 35 palabras reservadas de Python 3.12. No pueden emplearse como identificadores ordinarios. Se agrupan por uso para facilitar su consulta. [4]

Palabra(s)	Descripción
False / True / None	Falso, verdadero y ausencia de valor.
and / or / not	Conjunción, disyunción y negación lógica.
if / elif / else	Condición, condición alternativa y caso restante.
for / while	Iteración por elementos y repetición condicional.
break / continue	Salir del ciclo; pasar a su siguiente iteración.
def / return / lambda	Definir función, devolver resultado y crear función anónima.
class / pass	Definir clase; instrucción vacía.
try / except / finally	Proteger código, atender excepción y ejecutar limpieza final.
raise / assert	Lanzar excepción; comprobar una condición.
import / from / as	Importar módulo, seleccionar origen y asignar alias.
with	Administrar entrada y salida de un contexto.
in / is	Comprobar pertenencia e identidad de objetos.
del / global / nonlocal	Eliminar referencia y declarar alcance global o envolvente.
yield / async / await	Producir valores de generador; definir y esperar operaciones asíncronas.

Palabras contextuales en Python 3.12: match y case participan en patrones; _ es comodín en esos patrones; type inicia una declaración de alias de tipo. Fuera de esos contextos pueden ser identificadores. [4]

Cuestionario de la práctica continuación

4. ¿Qué operadores de asignación se pueden utilizar?

La asignación simple usa =. Las asignaciones aumentadas son +=, -=, *=, /=, //=, %=, **=, @=, &=, |=, ^=, >>= y <<=. Combinan la operación con una asignación; @= corresponde a multiplicación matricial cuando el tipo la admite. := es una expresión de asignación: asigna un valor y también lo produce dentro de una expresión. En el proyecto se utiliza -= para descontar existencias. [5]

5. ¿Cómo se realizan comentarios de línea y multilínea?

Un comentario empieza con # y termina al finalizar la línea. Para comentar varias líneas se escribe # al inicio de cada una. Las comillas triples delimitan cadenas, no un operador especial de comentario; una cadena al inicio de un módulo, clase o función puede ser su docstring. [4]

```
# Comentario de una línea
# Primera línea de un comentario extenso
# Segunda línea del comentario

def ejemplo():
    """Documenta la función ejemplo."""
    return 1
```

6. ¿Qué es una lista y cuáles son sus métodos?

Una lista es una secuencia mutable, ordenada y con elementos repetibles, por ejemplo ["P001", "P002"]. Sus métodos son: append(x), agrega al final; extend(it), incorpora elementos; insert(i, x), inserta; remove(x), elimina la primera coincidencia; pop(i), extrae; clear(), vacía; index(x), localiza; count(x), cuenta; sort(), ordena; reverse(), invierte; copy(), copia superficialmente. En la tienda, ventas es una lista y append() registra una venta nueva. [6]

7. ¿Qué es una tupla y cuáles son sus métodos?

Una tupla es una secuencia ordenada e inmutable, por ejemplo ("Código", "Nombre"). Puede contener elementos repetidos y referencias a objetos mutables. Sus métodos específicos son count(x), que cuenta coincidencias, e index(x), que devuelve la posición de la primera. En la interfaz se usan tuplas para definir columnas y configuraciones. [6]

8. ¿Qué son los conjuntos y cuáles son sus métodos?

Un set reúne elementos únicos y hashables sin un orden posicional definido. add() agrega; update() incorpora varios; remove() elimina y falla si el elemento falta; discard() elimina si existe; pop() extrae un elemento arbitrario; clear() vacía; copy() crea una copia. [3]

union(), intersection(), difference() y symmetric_difference() obtienen unión, intersección, diferencia y diferencia simétrica. Sus variantes intersection_update(), difference_update() y symmetric_difference_update() modifican el conjunto. issubset(), issuperset() e isdisjoint() comprueban relaciones. frozenset es inmutable y no ofrece métodos que cambien su contenido. Los conjuntos no fueron necesarios para almacenar los registros de esta aplicación. [3]

Cuestionario y referencias

9. ¿Qué estructuras de control utilizaste?

Se utilizó if para verificar la sesión, los permisos, las existencias y el pago; else para elegir rutas de ejecución, como cargar datos existentes o crearlos; y for para recorrer artículos, construir tablas y descontar unidades. Los bloques try/except manejan errores de entrada y de archivo; finally asegura la limpieza del archivo temporal. with delimita la escritura y garantiza el cierre del archivo. La interfaz usa el ciclo de eventos mainloop() de Tkinter; no fue necesario escribir un while manual.

10. Describe el procedimiento para utilizar una función

La función dinero(valor), definida en modelo.py con def, recibe un argumento, lo convierte a Decimal, rechaza valores negativos, no finitos o fuera de rango y devuelve el importe redondeado a dos decimales mediante return. Para usarla, se importa desde el módulo y se llama con el valor que se desea validar. Si el texto es inválido, la función lanza ValueError y la interfaz presenta el mensaje.

```
from modelo import dinero

precio = dinero("35.00")
pago = dinero("200")
# precio y pago son valores Decimal.
```

También se emplean sum() para sumar los importes del detalle, len() para obtener el siguiente folio y enumerate() para colocar campos en la interfaz. Estos ejemplos corresponden al código entregado.

Referencias bibliográficas

[1] Manual de prácticas de Lenguajes de Programación. Práctica 8: Programación con Python, pp. 1-5. Documento proporcionado para esta actividad.

[2] Python Software Foundation. General Python FAQ. https://docs.python.org/3.12/faq/general.html#what-is-python

[3] Python Software Foundation. Built-in Types. https://docs.python.org/3.12/library/stdtypes.html

[4] Python Software Foundation. Lexical analysis. https://docs.python.org/3.13/reference/lexical_analysis.html

[5] Python Software Foundation. Simple statements. https://docs.python.org/3.13/reference/simple_stmts.html

[6] Python Software Foundation. Data Structures. https://docs.python.org/3.13/tutorial/datastructures.html

[7] Python Software Foundation. tkinter. https://docs.python.org/3.13/library/tkinter.html

Las respuestas describen la sintaxis de Python 3.12, compatible con la documentación consultada para estos apartados. La implementación y los resultados se verificaron sobre los archivos entregados.